Indexing
— Chapter 14

2025 年 11 月

School of Computer Science

北京郵電大學

Outline

- Basic Concepts
- Ordered Indices
- B^+-Tree Index Files
- B-Tree Index Files
- Hashing

目标

- 建立索引，提高数据库**访问速度**
- 索引对 select/update/insert/delete 的影响
  - 索引类型
  - select, insert, delete, update，索引管理
  - 复合索引左前缀原则

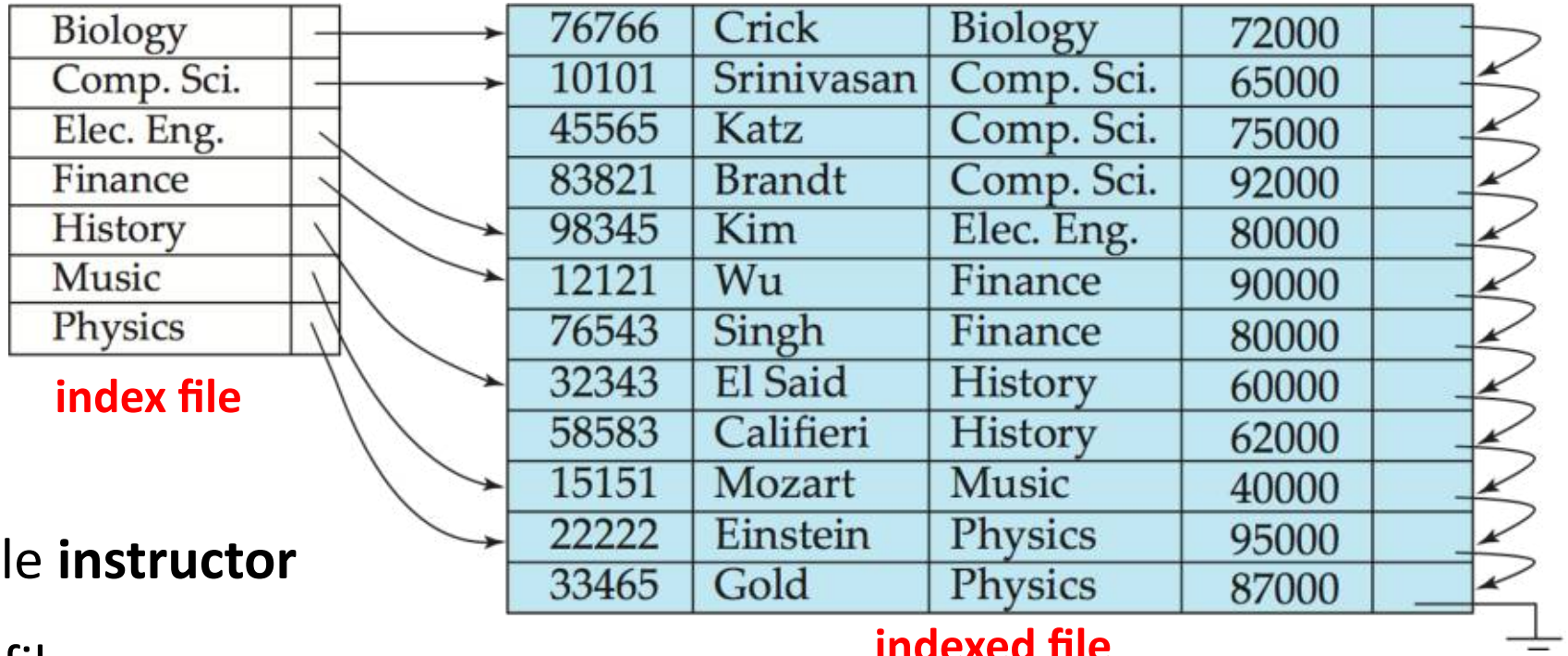
14.1 Basic Concepts

- **问题： Locate** tuples/records **quickly**
- **Indexing** (索引) mechanisms
  - speed up access to desired data
  - e.g. instructor (ID, name, dept_name, salary)

    index on instructor (dept_name)

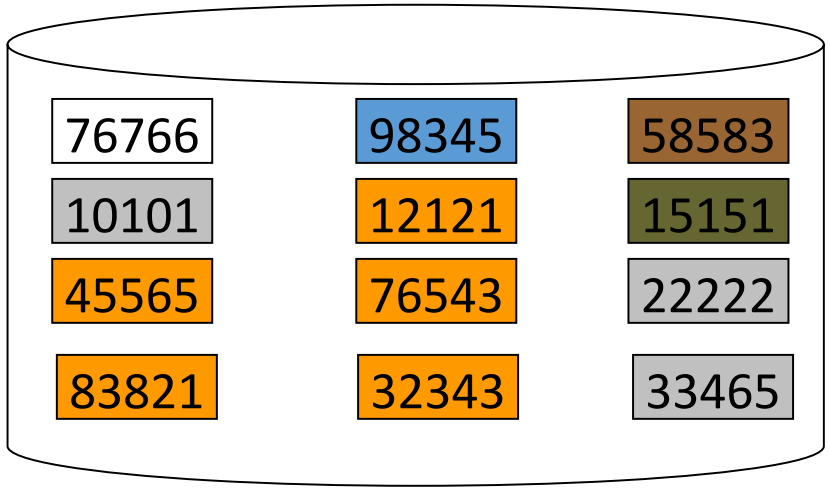
    dept_name → **physical address** of record in DB file instructor
- **Def.: Search Key**
  - **an attribute or a set of attributes** used to look up records
  - e.g. dept_name

instructor(ID, name, dept_name, salary)



- DB indexed file **instructor**

  and its index file



Note: the file **instructor** is logically a sequential file,

but its records may be stored non-contiguously or non-ordered on disk

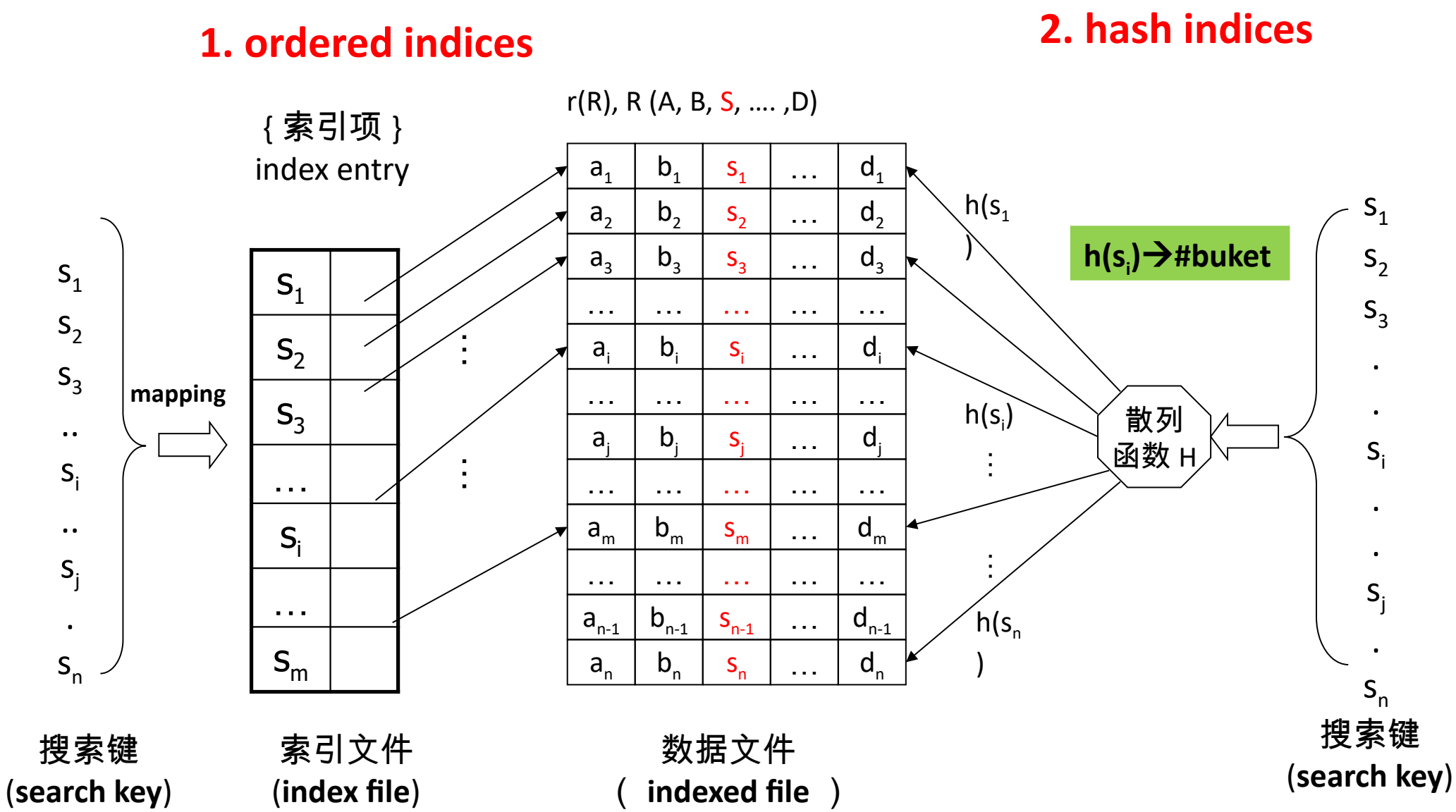
Basic Concepts (cont.)

- **Def 1:** An **index file** consists of **records** (called **index entries**) of the form
  - Index files are typically much smaller than the original file

search-key	pointer

- **Def 2:** **Indexing**
  - **mapping** from search-key to storage locations of the **records** **in disk**,

    search-key → storage location


1. ordered indices
2. hash indices
{ 索引项 }
index entry
r(R), R (A, B, S, …. ,D)
mapping
h(s1)
h(si)
h(sn)
h(si)→#buket
散列
函数 H
搜索键
(search key)
索引文件
(index file)
数据文件
(indexed file)
搜索键
(search key)

Basic Concepts (cont.)

- **Two indices**
  - **1. ordered indices:**
    search key and address of the records are stored in **sorted order**
  - **2. hash indices:**
    hash function map search key to the address of the records
    - the records are stored in **buckets**
    - the number of buckets is the address of the records
    - determined by hash function
- **Access types**
  - 等值查询， records with a specified value, e.g. ID=1254
  - 比较查询， e.g. salary>10000
  - 范围查询， e.g. salary between 10000 and 20000

14.2 Ordered Indices

- DBS files with indexing mechanism includes:
  - **indexed** file, in which **data records** are stored
  - **index file**, in which **index entries** are included
- The **indexed file** can be organized
  - **sequential** file, **heap** file, **hash** file, **clustering** file
- The **ordered index** (in index file)
  - the index entries are stored in sorted order,
    i.e. the order of the search key
  - e.g. the index file is sorted by **dept_name**

    索引项的排列顺序与搜索键的排列顺序一致

1. Primary Indices

- **Def 1**: **The primary/clustering index** (in the index file)
  - The search key of the index specifies the sequential order of the indexed file
  - The indexed file is a sequential file
  - e.g. (*dept_name*, address of records) defines the same orders of the records as that in sequential indexed file **instructor**

索引文件搜索键所规定的顺序与被索引的顺序文件纪录顺序一致

聚集索引 vs 主索引

- **聚集索引**
  - 索引文件搜索键所规定顺序与被索引文件纪录顺序一致
- **主索引**
  - 建立在主键（主属性上）的索引
- 当表定义了主键后，DBMS 自动为该表在主键上建立聚集索引，该索引又是主索引
  - 主索引一定是聚集索引，但聚集索引不一定是主索引
  - **1** 个表上只能建立 **1** 个聚集索引，也只能有 **1** 个主索引
  - 如果表没有定义主键，不会有主索引；但可以为该表建立聚集索引

1. Primary Indices

- 对比分析
  - 1. 在没有定义主键的关系表中插入数据：

    在关系表所在数据库文件中，各个元组 / 行 / 记录顺序一般是无序，取决于数据插入顺序—— heap 文件
  - 2. 在有主键的关系表中插入数据：

    数据库文件是有序的，按照主键顺序排列

2. Secondary Indices

- **Def 2: Secondary index / non-clustering index**
  - a search key specifies an order different from the sequential order of the file
  - secondary indices have to be dense indices
  - e.g. secondary index on **salary** field of **instrutor**
- **Def 3: Index-sequential file** (索引顺序文件)
  - ordered sequential file with a primary/clustering index on the search key

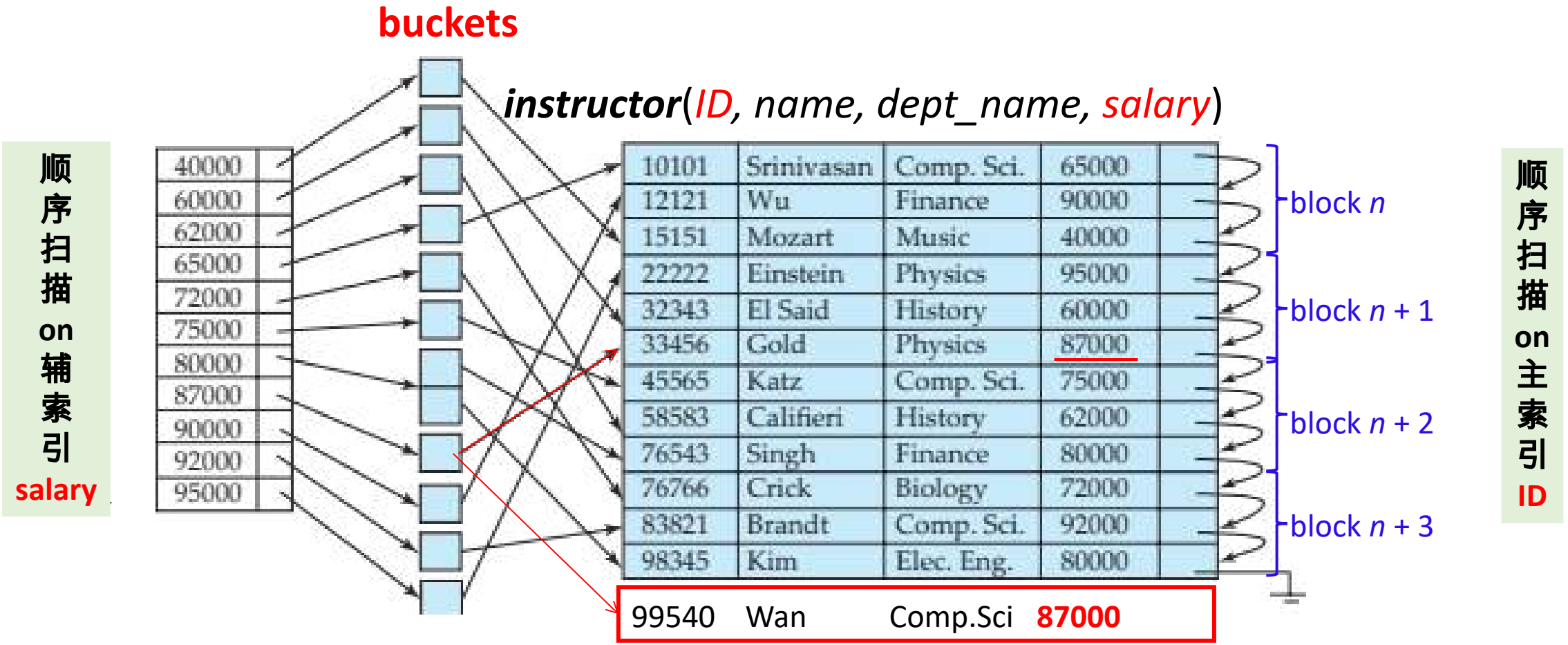


Fig. 11.6 Secondary index on **salary** field of **instructor**

- 1. Secondary indices have to be **dense**
- 2. Index record points to a **bucket** that contains pointers to the actual records with that particular search-key value.

3. Dense and Sparse Indices

- **Def 4: Dense index**
  - the index record appears for **every** search-key value in *the indexed file*
  - **each value** of search-key corresponds to **an index entry** in *the index file*
  - e.g. dense index on dept_name, with **instructor** file sorted on *dept_name*
- **Def 5: Sparse Index**
  - index file contains index entries for **only some** search-key values

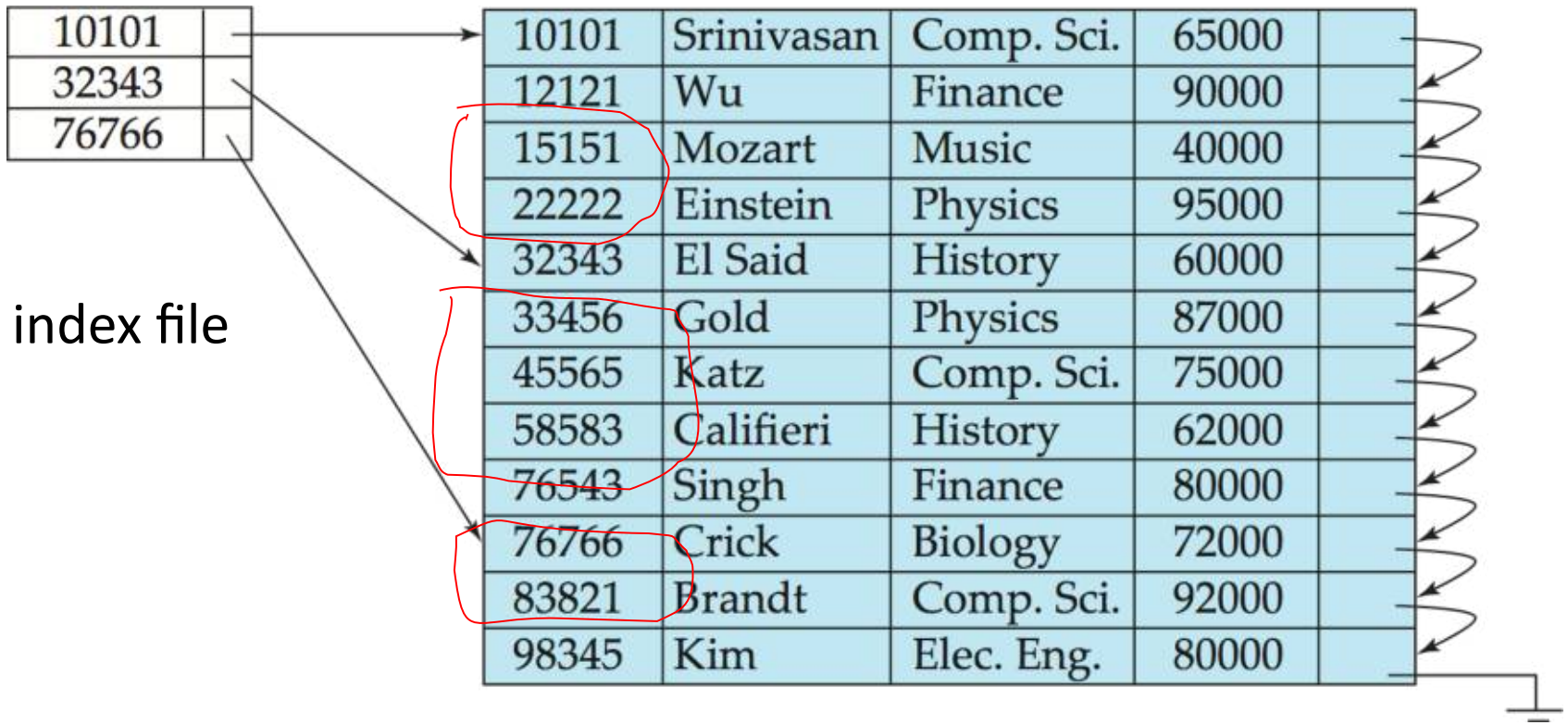


Fig.11.3 Sparse index for the file **instructor**

3. Dense and Sparse Indices

- For **dense primary index**, the index record contains the search-key value
  - a pointer to **the first** record according to that search-key value
  - **the rest** of the records with the **same** search-key value would be stored sequentially after the first record
  - e.g. search-key value **Comp.Sci** corresponds to **three** records
- For **sparse index**, to locate a file record with the search-key value *K*
  - **find** index entry with the largest search-key value ≤ *K*
  - **search** file sequentially starting at the record to which this index entry points

4. Primary vs Secondary Indices

- **1. Updating indices** imposes overhead on DB modification
  - when a file is modified, every index on the file must be updated
- **2. Sequential scan** using **primary index** is **efficient**
- **3. Sequential scan** using **secondary index** is **expensive**
  - each record access may fetch a new block from disk
    - 读磁盘数据时，磁盘读写头总移动距离较长，访问时间长

Example 1

- The data file **instructor** is stored in order of *ID*, the primary index
  - sequential scanning on *ID* of the **first six** tuples/records, i.e. 10101-33456, will fetch block *n* and block *n*+1
  - sequential scanning on *salary* of the **first six** tuples/records, i.e. 40000-75000, will fetch block *n*, *n*+1 , *n*+2 , *n*+3
- Example:
  - 1. select *
    from *instructor*
    where ***ID*** between 10000 and 40000
    2. select *
    from *instructor*
    where ***salary*** between 40000 and 80000

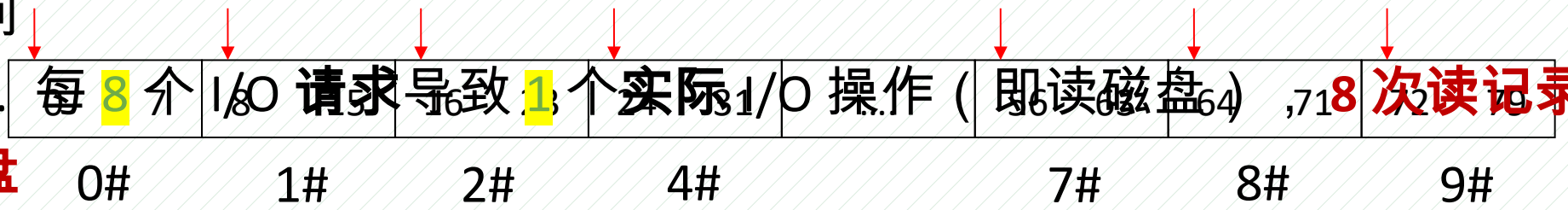
Example 2

- A DB file is made up of the **128-byte** fix-sized logical records
  It is stored on disk in unit of block that is of **1024 bytes** in size.
- The size of the file is **10240 bytes**
- Physical I/O operations transfer data on disk into an DBMS buffer in main memory,
  in terms of **1024-byte** block
- **Problem**:
  If a transaction issues *read* requests in a sequential access manner,
  what is the percentage of *read* requests that result in I/O operations?

Example 2

- The file contains 10240/128=80 records
- A block holds 1024/128=8 records
- The file is stored in 10240/1024=10 blocks
- percentage = 10/80=1/8=0.125

1. 按照记录存储顺序，依次访问每个记录时，每个记录的访问都对应一个 I/O 请求
2. 每次 I/O 访问， DBMS 将 1 个 block 中的 8 个记录读到内存 buffer 中。
   当后续再访问这 8 个记录时，直接从 buffer 中读，无需访问 disk ，即无需 I/O 访问



Multi-level Indices

- **问题**： The index file may be very large, and cannot be entirely kept in memory

  If primary index does not fit in memory, access becomes expensive.

- **Solution**: treat primary index kept on disk as a sequential file and construct a sparse index on it.
  - **outer index** – a sparse index of primary index file
  - **inner index** – the primary index file
- If even outer index is too large to fit in main memory, yet another level of index can be created, and so on.

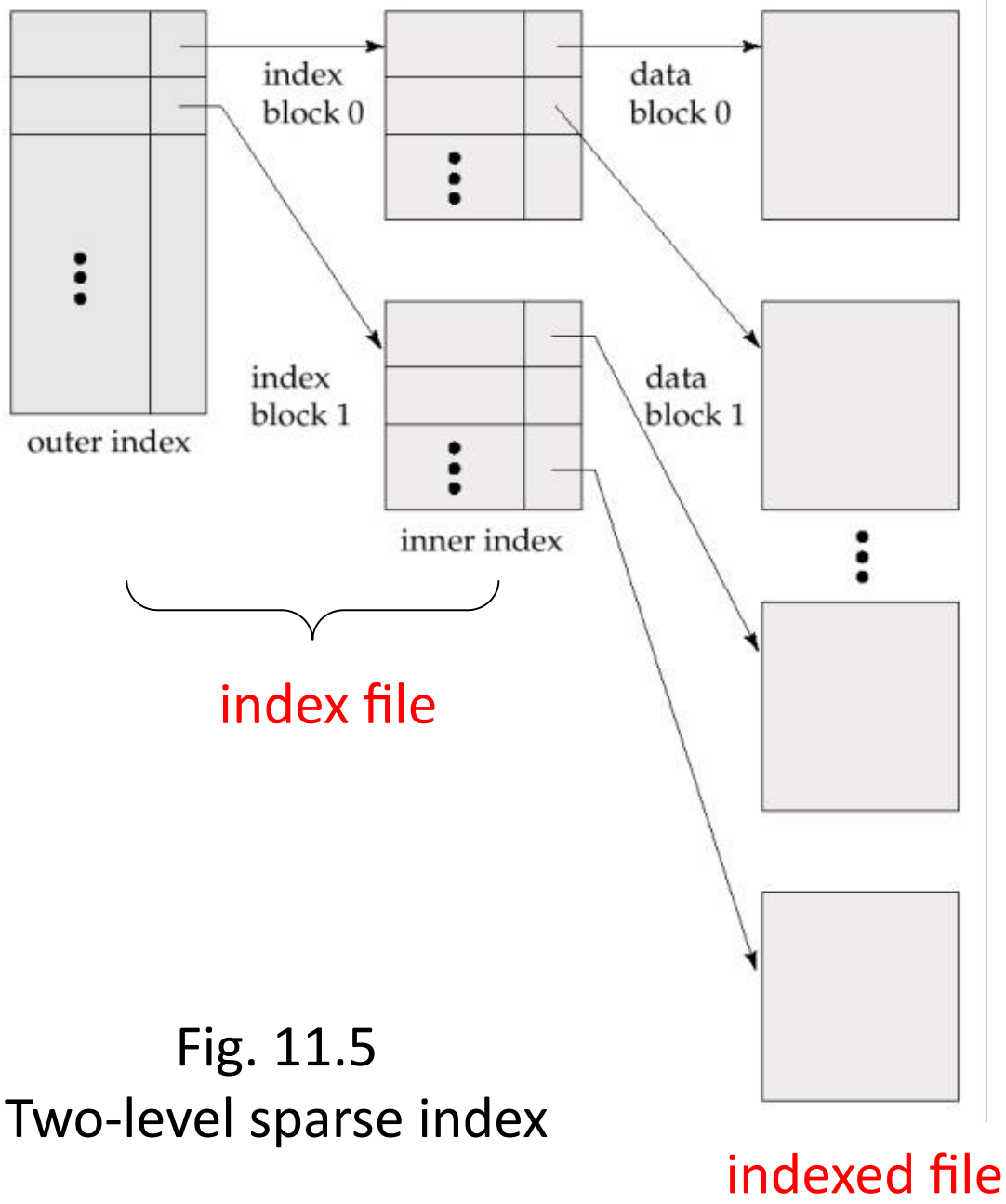


Fig. 11.5
Two-level sparse index

- B^+-tree indices and B-tree indices are two **efficient** multi-level indices, widely used in DBS

14.3/14.4 B⁺-Tree/B-Tree Index Files

- **B-tree: Wikipedia**
  - Proposed in 1970, Bayer R.
  - B tree is a self-balancing tree structure that
    1. keeps data stored
    2. allows sequential accesses, insertions, and deletions
       in logarithmic time
- **B⁺tree:** B tree 演化产生
  - B⁺tree indices are an **alternative** to the **indexed-sequential file**
  - B⁺tree used multi-level index, with advantages

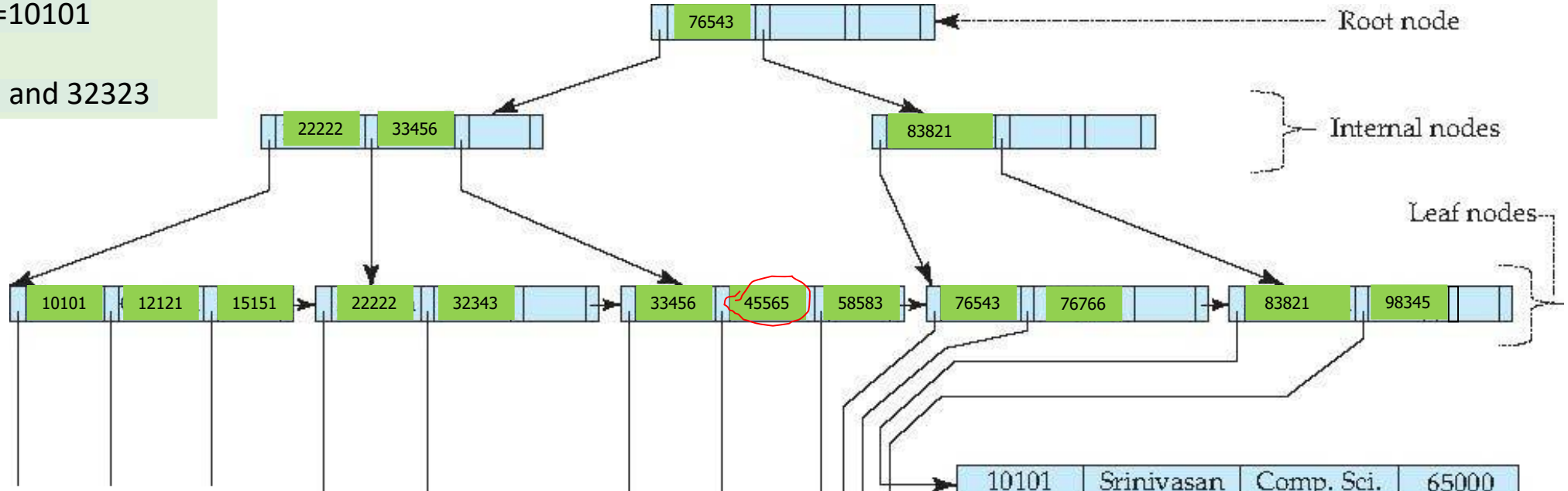
    automatically reorganizes itself, for insertions and deletions

文件记录直接存储在叶节点

1. B+-Tree (n=4) Index Sequential File
Primary/Clustered index on ID

n: degree or branching factor

1. 等值查询： ID=10101
2. 范围查询，
   ID between 10101 and 32323



10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	80000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
58583	Califieri	History	60000
76543	Singh	Finance	80000
76766	Crick	Biology	72000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000

- create table *instructor*
  (*ID* integer, primary key;)

Or:

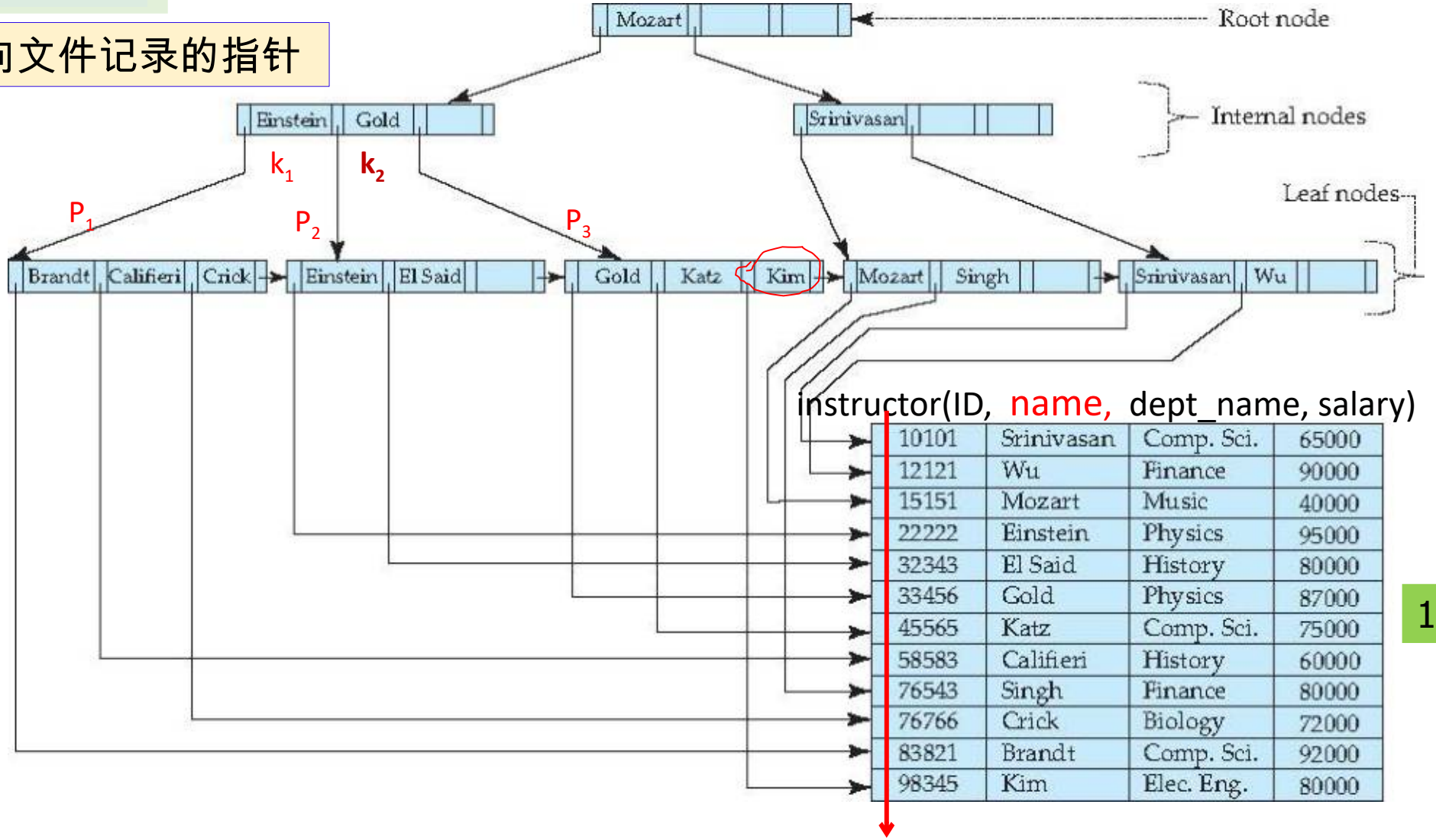
- create clustered index
  ID Index on *instructor*(*ID*)

1. 等值查询， name= ‘ Gold ’
2. 范围查询，
name between ‘Gold’ and ‘Wu’

叶节点存储指向文件记录的指针

2. B+-Tree (*n*=4)

secondary/nonclustered index on *name*



instructor(ID, name, dept_name, salary)

10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	80000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
58583	Califieri	History	60000
76543	Singh	Finance	80000
76766	Crick	Biology	72000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000

12 records

B^+-tree Properties

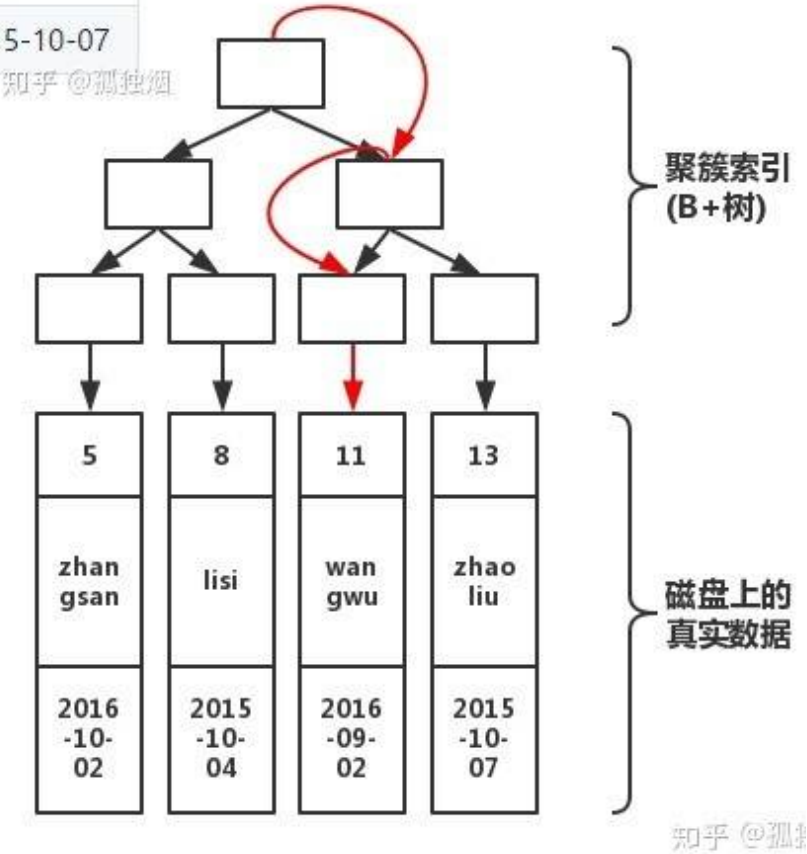
- **Def.1**: B^+-tree is a rooted tree, with the parameter degree (branching) factor n
  - All paths from root to leaf are of the same length: balanced tree
- 1. Each internal node (not root or leaf) has between $\lceil n/2 \rceil$ and n children,
  - e.g. n=4
- 2. Each leaf node has between $\lceil (n-1)/2 \rceil$ and n–1 search key values
  - e.g. n=4, 2~3 values
- 3. Special cases for the root
  - if the root is not a leaf, it has at least 2 children
  - if the root is a leaf, it has between 0 and (n–1) values.

Examples

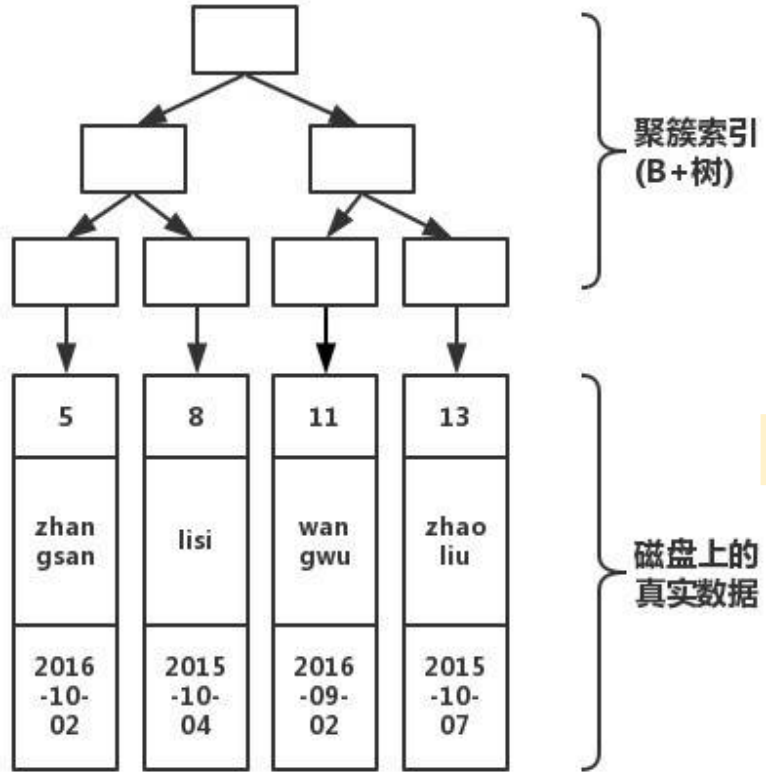
pId	name	birthday
5	zhangsan	2016-10-02
8	lisi	2015-10-04
11	wangwu	2016-09-02
13	zhaoliu	2015-10-07

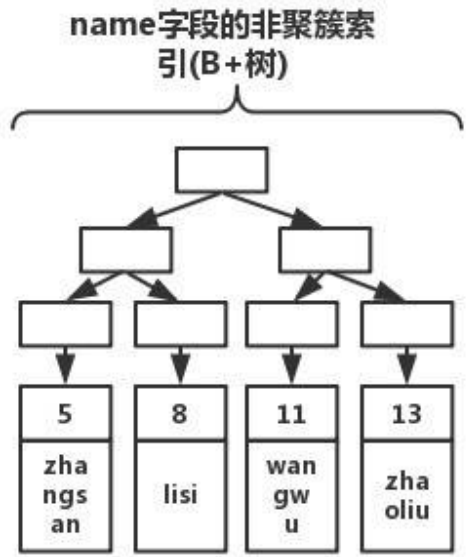
1. 建立在主键 pid 上的聚集索引

根据主键 pid=11 查找记录



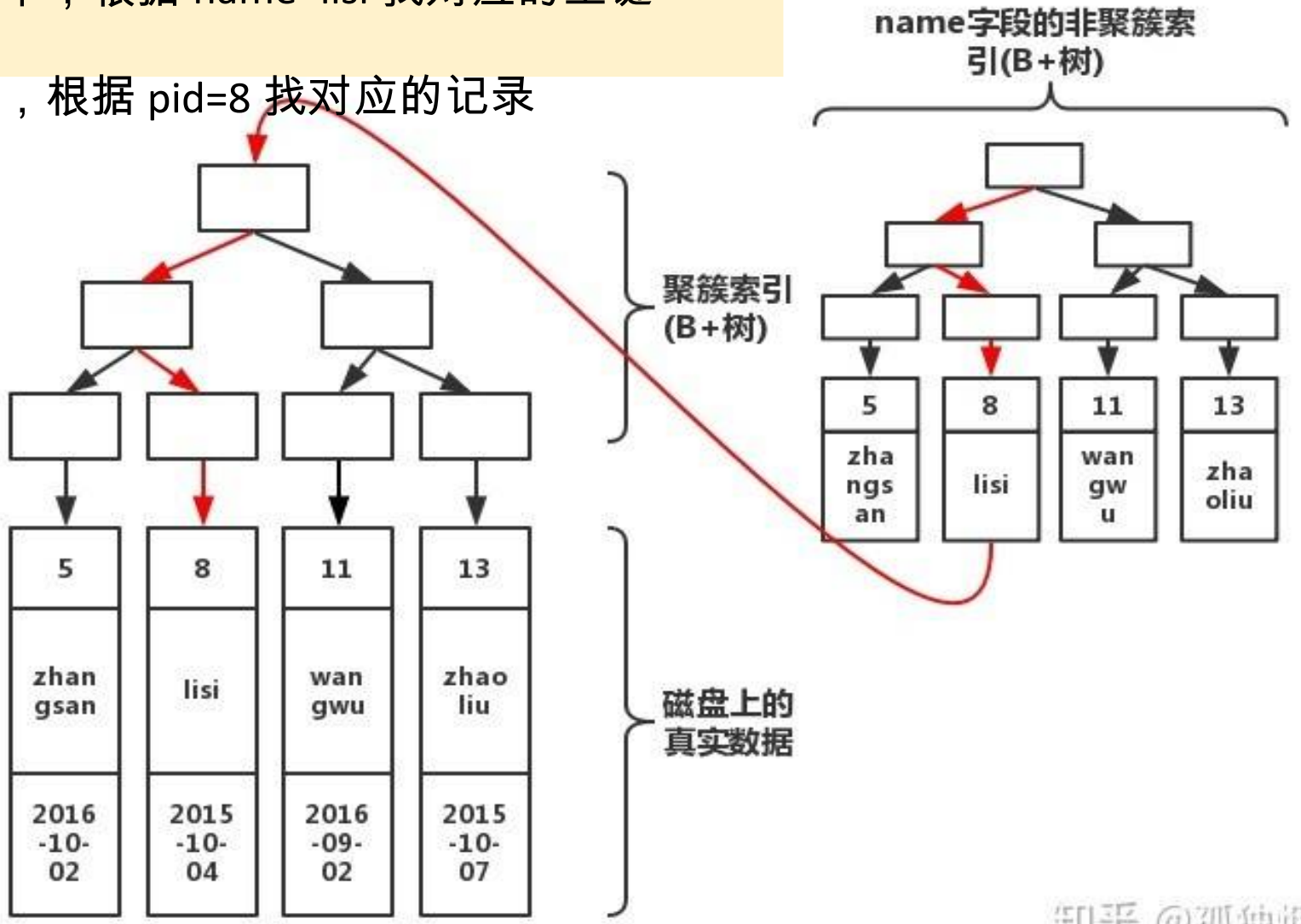
2. 建立在 name 上的非聚集索引





叶结点存 <name,pid>

根据 name 查找记录：
步骤 1：非聚集索引中，根据 name=lisi 找对应的主键 pid=8
步骤 2：聚集索引中，根据 pid=8 找对应的记录



B+-Tree Node Structure

- The search-keys in a node are ordered
- **Typical node**

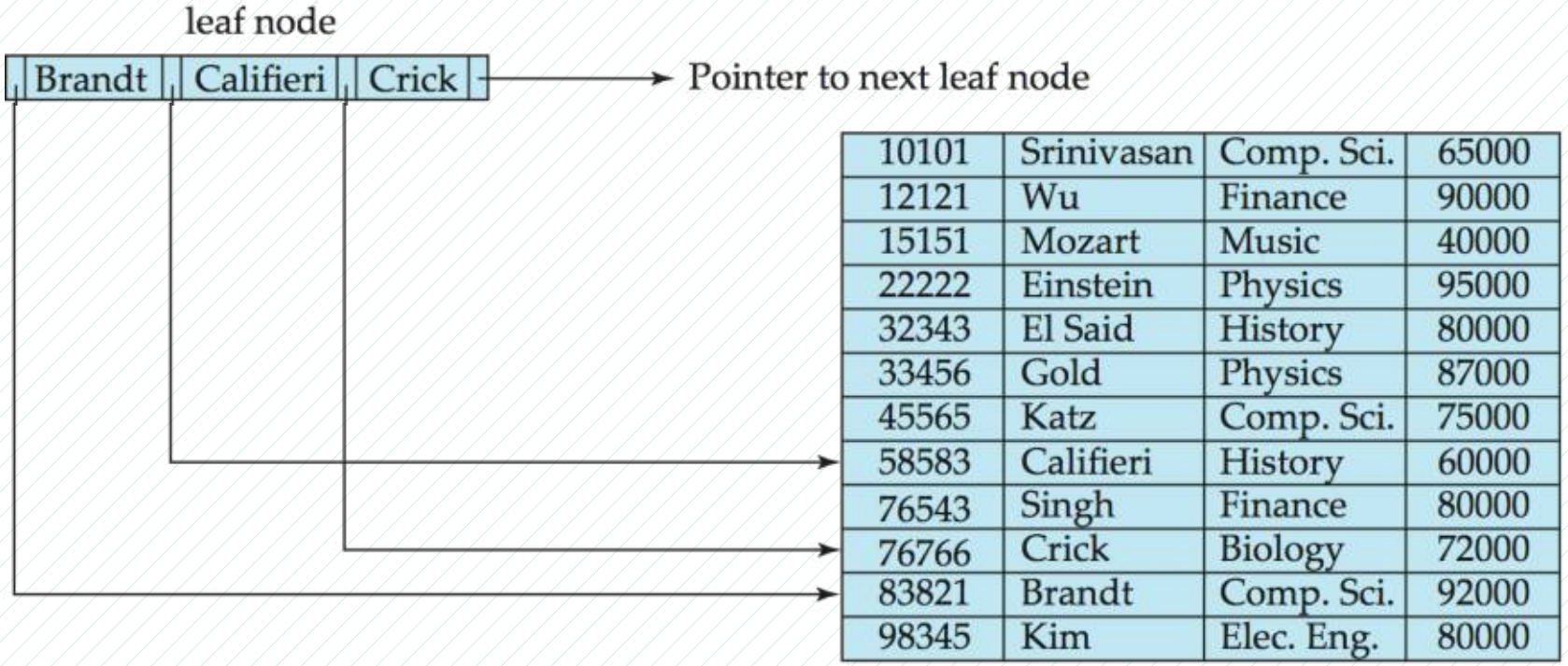
  - K_i are the search-key values
  - P_i are
    - 1. pointers to children (non-leaf nodes)
    - 2. pointers to records or buckets of records (leaf nodes).

$$K_1 < K_2 < K_3 < \ldots < K_{n-1}$$

Leaf Nodes in B+-Trees

- **1. leaf node ： Properties**
- P_i points to a record with search-key value K_i, i <n–1,
- P_n points to the next leaf node
- If L_i, L_j are leaf nodes, such that i < j,

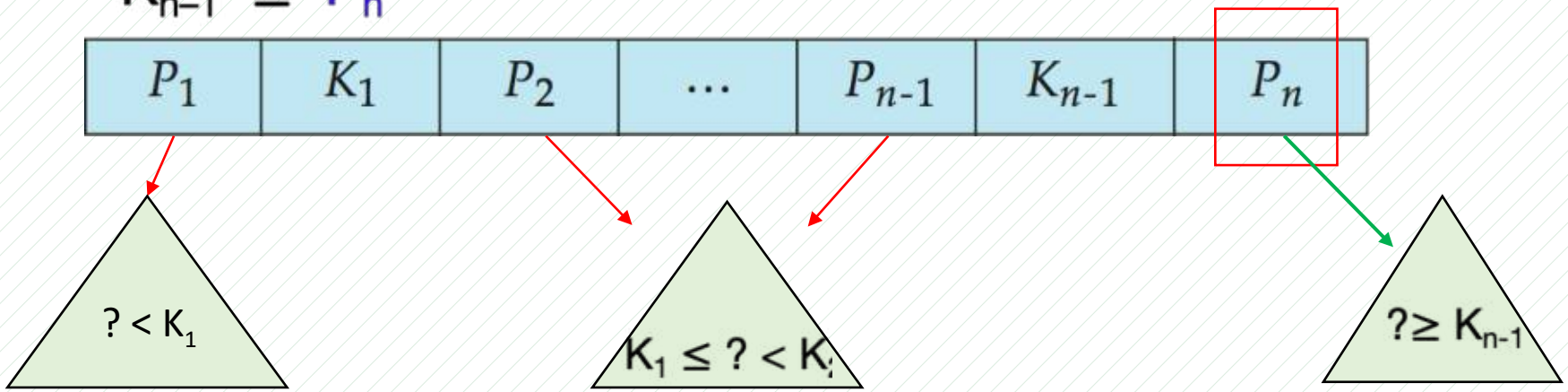
  L_i's search-key values are less than or equal to L_j's search-key values



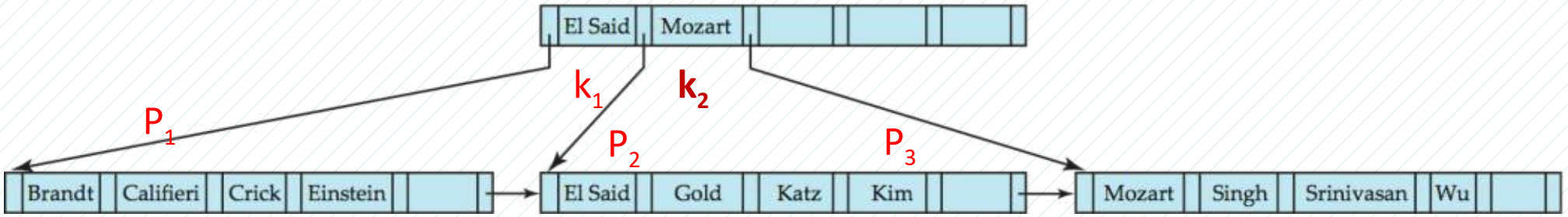
10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	80000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
58583	Califieri	History	60000
76543	Singh	Finance	80000
76766	Crick	Biology	72000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000

Non-Leaf Nodes in B+-Trees

- **2. Non-leaf nodes**: form a **multi-level sparse** index on leaf nodes.
  - **non-leaf node** with n pointers:
    - 1. P_1: The search-keys to which P_1 points have values
      $P_1 < K_1$
    - 2. P_i: the search-keys to which P_i ($2 \le i \le n-1$), points have values
      $K_{i-1} \le P_i < K_i$
    - 3. P_n: The search-keys to which P_n points have values
      $K_{n-1} \le P_n$



Example of B⁺-tree



B⁺-tree for *instructor* file (n = 6)

- Leaf nodes must have between 3 and 5 values
  - $\lceil (n-1)/2 \rceil$ and $n-1$, with $n = 6$
- Non-leaf nodes other than root must have between 3 and 6 children
  - $\lceil (n/2 \rceil$ and n with $n = 6$
- Root must have at least 2 children

Observations about B⁺-trees ： B⁺-trees Performance

- **优势**：
  - Since the inter-node connections are done by **pointers**, **logically** close blocks need not be **physically** close.
  - The non-leaf levels of B⁺-tree form a hierarchy of **sparse indices**.
  - If there are *K* search-key values, the height is no more than $\lceil \log_{\lceil n/2 \rceil}(K) \rceil$

Non-Unique Keys

- **Non-Unique Keys**:

  If a search key a_i is not unique,

  create an index on a composite key (a_i, A_p), which is unique

  - A_p could be a primary key, i.e., record ID,

    or any other attribute that guarantees uniqueness

- Search for $a_i = v$ by a range search on composite key

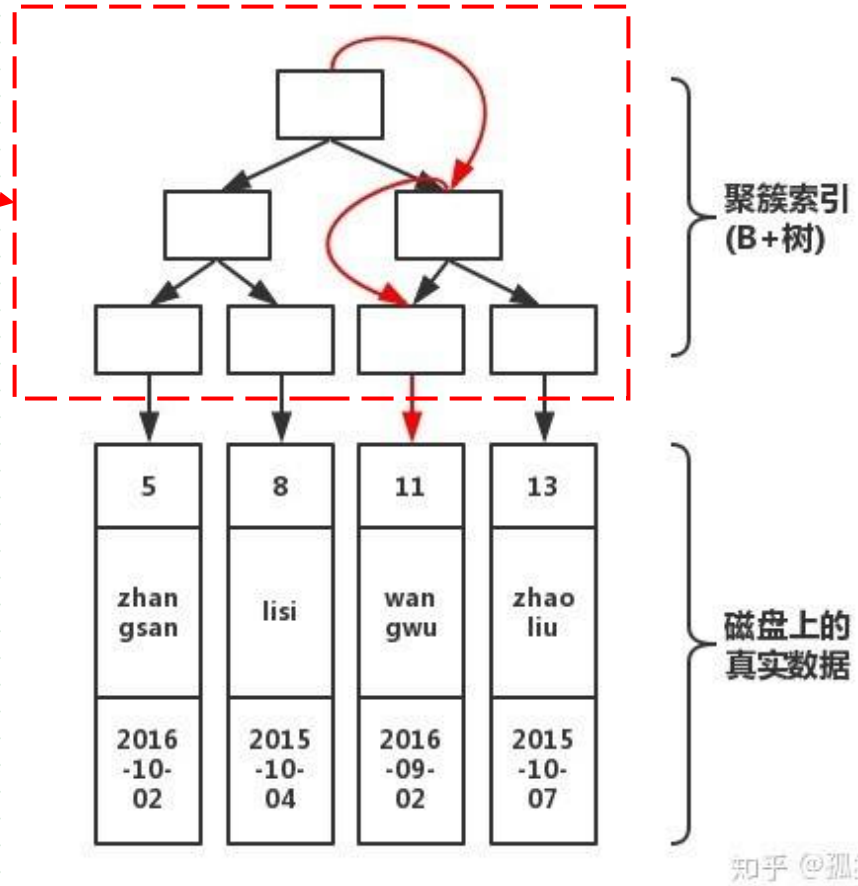
- More I/O operations are needed to fetch the actual records
  - If the index is clustering, all accesses are sequential
  - If the index is non-clustering, each record access may need an I/O operation

Queries on B+-Trees (Cont.)

Note: MySQL 数据库中，索引树 n<=100 ，一张表可支持 10 亿行数据，
但为了控制 B+-tree 索引深度，一般不超过 2000 万行

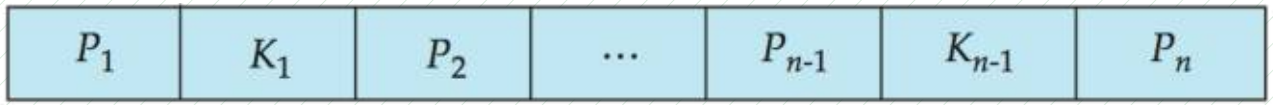
- Why?
  - B+-tree索引（非叶节点）需要全部调入内存
  - B+-tree深度与索引树中非叶节点数目S(B+-tree)所占内存空间取决于
    - 表中不同search-key数目（instructor表ID数目）
  - 假设系统分配给数据库系统可用内存大小为M，要求：

    $S(B+\text{-tree}) \leq M$
  - 根据内存M、索引树节点大小*n*值确定关系表的行数



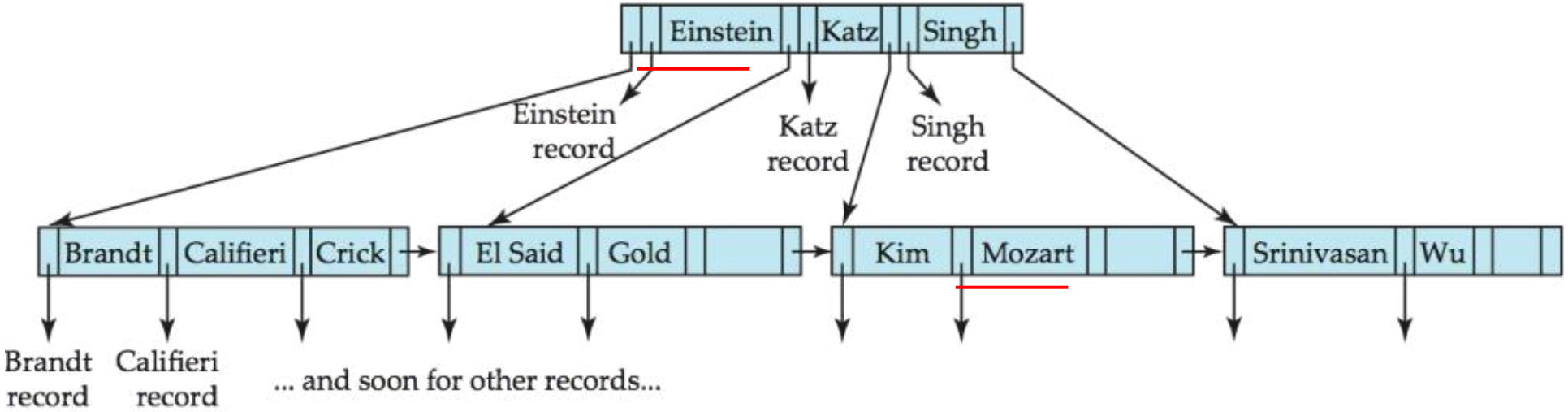
B-Tree Index Files

- **B-tree** ： allows search-key values to appear **only once**
  - **eliminates redundant** storage of search keys.
- **Non-leaf nodes** ： Search keys appear **nowhere else**
  - an additional pointer field for each search key must be included.
  - pointer Bi are the bucket or the record pointer to Leaf node ：

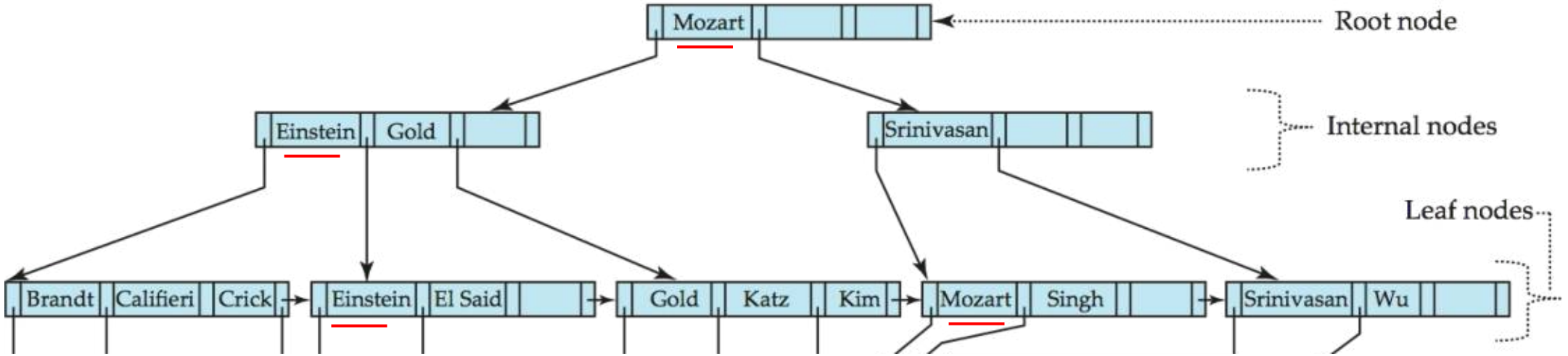


(a)

(b)



B-tree (above) and B+-tree (below) on same data



B-Tree Index Files (Cont.)

- **Advantages**:
  - **less** tree nodes
  - **find** search-key value **before** reaching leaf node
- **Disadvantages**:
  - **Only small fraction** of all search-key values are found early
  - **Fan-out** is **reduced,** as non-leaf nodes are larger.
  - Insertion and deletion are more **complicated**

14.5 Hash Indices

- The records are stored in **buckets**
  - **bucket** is a **unit** of storage containing records
    - a bucket is a disk block
  - **bucket** can be obtained from search-key using **hash function**
- **Hash function** *h*
  - a function from search key ***K*** to bucket addresses
    i.e. the addresses of the records
  - Entries with different search keys mapped to the same bucket
  - The entire bucket searched sequentially to locate an entry.

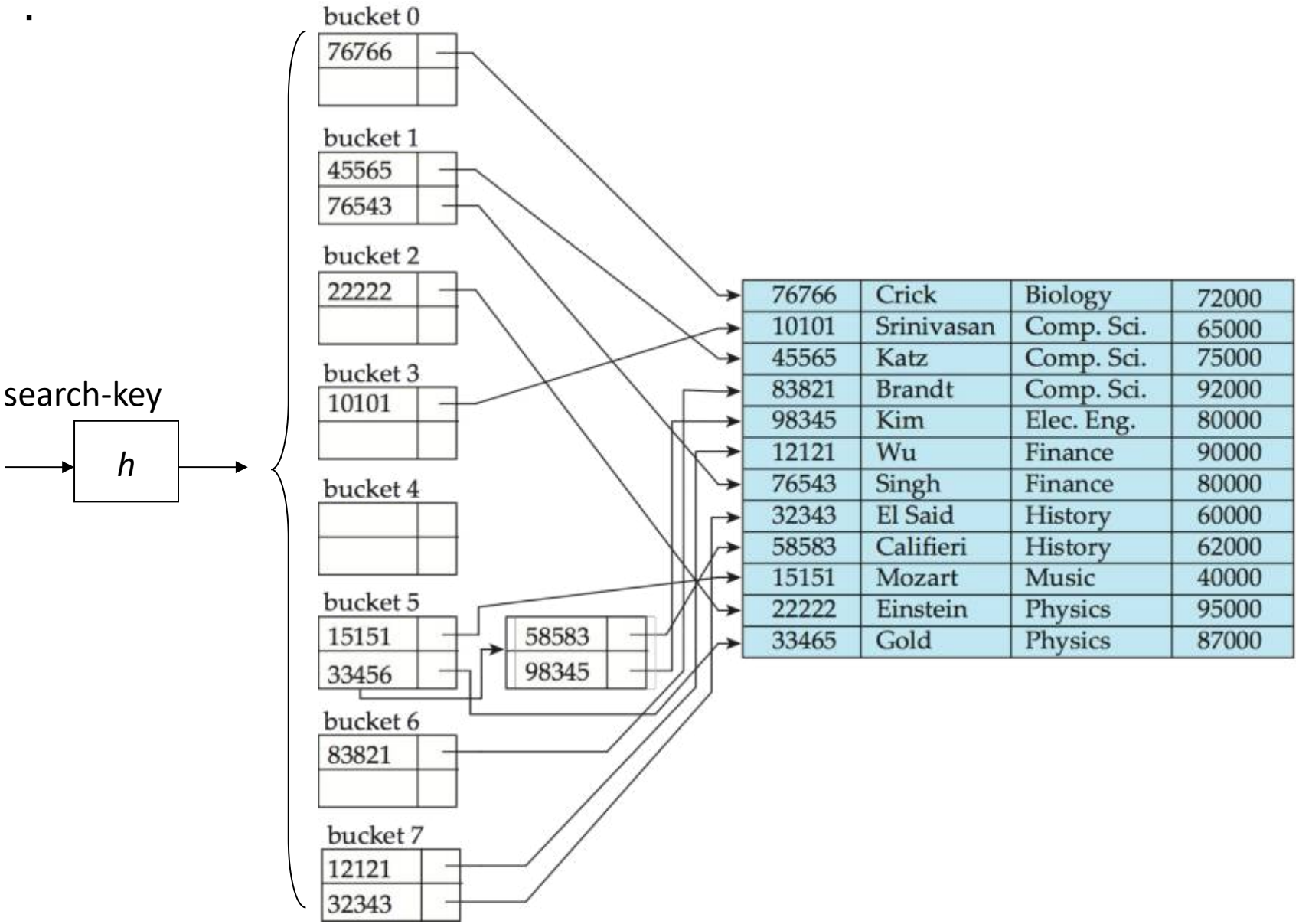
Hash Indices

- **hash index【哈希索引】**
  - buckets store **entries** with **pointers** to records

    search key→pointer to records
- **hash file organization【哈希文件组织】**
  - buckets store **records** of the hash file
  - obtaining the bucket of a record from search key using **hash function**:

    search key→records
- **Static hashing**
  - hash function *h* cannot be modified
- **Dynamic hashing**
  - hash function *h* can be modified dynamically

■ **Example** : **hash index** on attribute **ID** of *instructor*



Example 1: Hash File Organization

- Hash the **instructor** file, using *dept_name* as key (next slide.)
  - There are 10 buckets,
  - The binary representation of the *i* th character is the integer *i.*
  - The hash function returns the **sum** of the binary representations of the characters modulo 10
    - **e.g**. h(Music) = 1 h(History) = 2
      h(Physics) = 3 h(Elec. Eng.) = 3

Example 1: Hash File Organization

- Hash the *instructor* file, using *dept_name* as key.

bucket 0

bucket 1

15151	Mozart	Music	40000

bucket 2

32343	El Said	History	80000
58583	Califieri	History	60000

bucket 3

22222	Einstein	Physics	95000
33456	Gold	Physics	87000
98345	Kim	Elec. Eng.	80000

bucket 4

12121	Wu	Finance	90000
76543	Singh	Finance	80000

bucket 5

76766	Crick	Biology	72000

bucket 6

10101	Srinivasan	Comp. Sci.	65000
45565	Katz	Comp. Sci.	75000
83821	Brandt	Comp. Sci.	92000

bucket 7

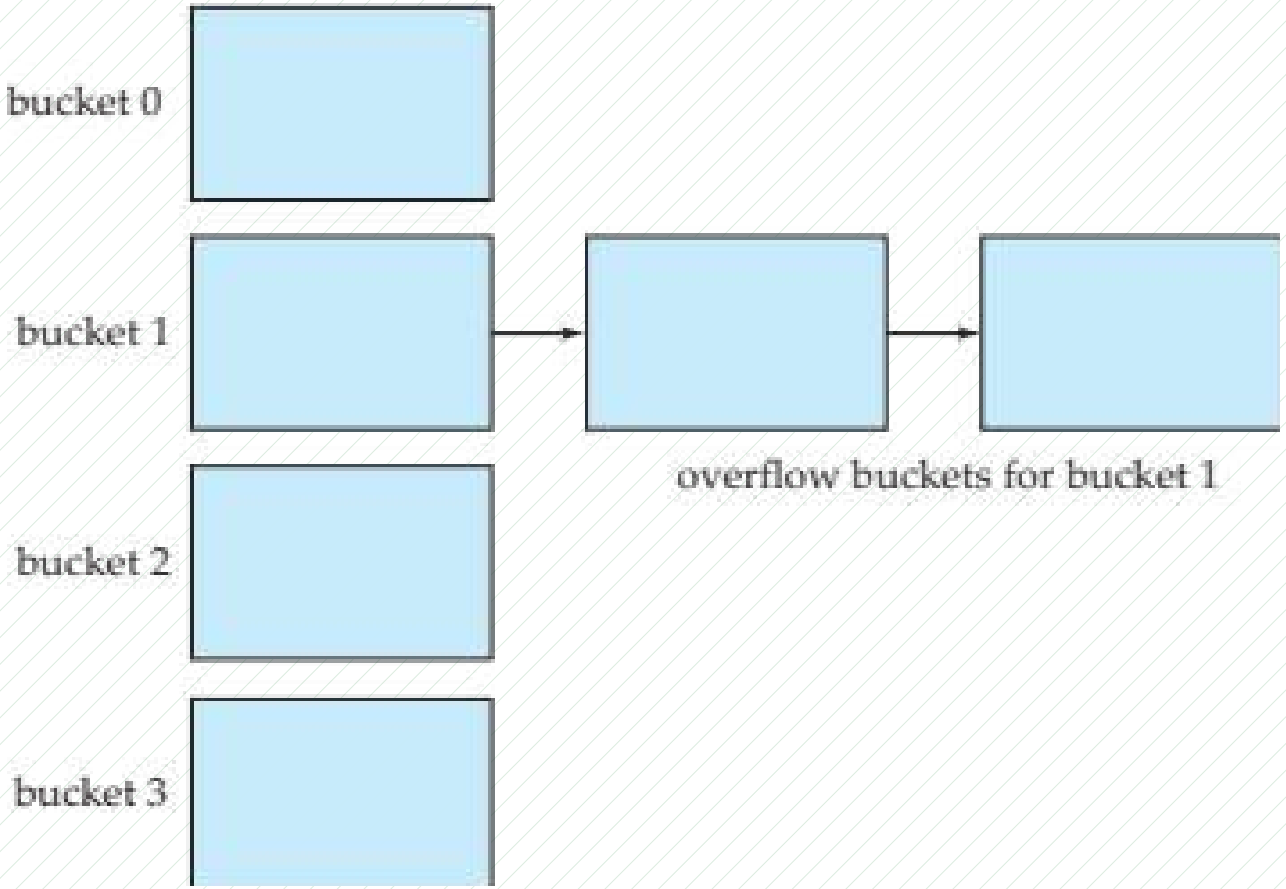
Bucket Overflows

- **Bucket overflow:** it occurs because of
  - insufficient buckets
  - non-uniform distribution of records due to:
    - multiple records have same search-key value
    - hash function produces non-uniform distribution of the key values
- The probability of bucket overflow can be reduced,
  but cannot be eliminated;
  it is handled by using **overflow buckets**

Handling of Bucket Overflows (Cont.)

- **Def: Overflow chaining**

  the overflow buckets of a given bucket are chained together.



Ordered Indexing VS Hashing

- Expected Queries:
  - Hashing is better at retrieving records having a specified value of the key, e.g. ID=10121
    - 等值查询
  - If range queries are common, e.g. age between 17 and 25, the ordered indices are to be preferred
    - **Hashing index 不支持 range query**

14.6 Multiple-Key Access

- **Motivation : select** *ID*

  from *instructor*

  where *dept_name* =Finance **and** *salary* = 80000

- Possible strategies using indices on single attributes:

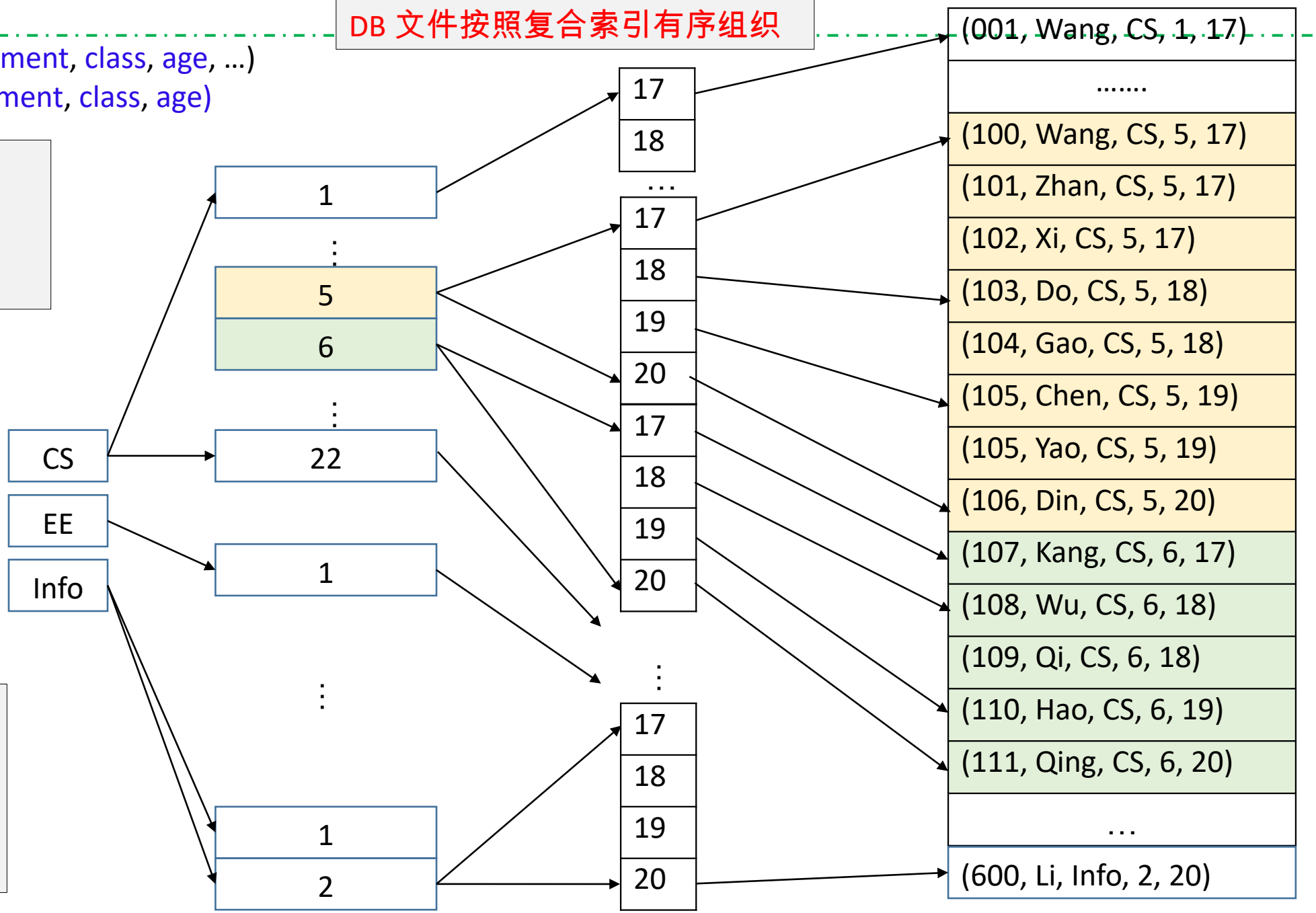
  1. Use **index** on ***dept_name*** to find instructors with department name Finance;
     Then test *salary = 80000*
  2. Use **index** on ***salary*** to find instructors with a salary of $80000;
     Then test *dept_name* = "Finance".
  3. Use index on ***dept_name*** to find pointers to all records pertaining to Finance department. Similarly use index on ***salary***.
     Take **intersection** of both sets of pointers obtained.

DB 文件按照复合索引有序组织

Student(ID, name, department, class, age, …)
composite index(department, class, age)

满足左前缀的查询，可以利用索引快速
seek ：
depart=CS and class,
depart=CS

不满足左前缀的查询，无法利用索引，只能 scan 整个文件：
class=5 and age=20,
age=20



Indices on Multiple Keys

- **Composite search keys** are search keys containing more than one attribute
  - e.g. (*dept_name, salary*)
  - e.g. create index Mutiple-index on takes (course_id, semester, year)
- **Lexicographic ordering**（字典顺序）:
  $(a_1, a_2) < (b_1, b_2)$ if either
  - $a_1 < b_1$, or
  - $a_1 = b_1$ and $a_2 < b_2$

复合索引左前缀原则

- Suppose we have an index on combined search-key

  (*dept_name, salary*)

- With the where clause

  where ***dept_name =*** “Finance” and ***salary =*** 80000

  index on (*dept_name, salary*) to fetch records with both conditions.

- Can also efficiently handle

  where ***dept_name =*** “Finance” and ***salary <*** 80000

- Can also efficiently handle

  where ***dept_name =*** “Finance”

复合索引左前缀原则

- But cannot efficiently handle
  - 1. where ***dept_name <*** “Finance” and ***salary =*** 80000
    - may fetch many records that satisfy the first but not the second condition
    - the index is not used efficiently
  - 2. where ***salary =*** 80000
    - the index is not used for query

14.7 Creation of Indices

- Indices on primary key created automatically
- Indices can speed up **lookups**, but impose **cost** on **updates**
  - index tuning assistants supported to help choose indices, based on query and update workload

自治数据库：
1. 具有自调整、自优化能力
2. 采用 AI 、机器学习技术

Index Definition in SQL

- Create an index

  create index <index-name> **on** <relation-name> (<attribute-list>)

  Example ： **create index** *b-index* **on** *branch* (branch_name)

  Example ： **create index** takes_pk **on** *takes* (ID,course_ID, year, semester, section)

- Drop an index ：

  drop index takes_pk

- Use **create unique index** to indirectly specify and enforce the condition that the search key is a candidate key

强制使用 / 不使用索引 in SQL Server

- 强制使用

```
select*
from  tbMRODataa  with (index=timeide)
where TimeStamp='2016-07-19T10:00:03.840'
```

- 强制不使用

```
alter index timeide on tbMRODatanew
disable
select*
from tbMRODatanew
where  TimeStamp='2016-07-19T10:00:03.840'
```

掌握：适宜建立索引的属性

- 对 select 查询，针对 select-from-where-groupby-having 操作，
  在以下属性上建立聚集或非聚集索引，有利于提高访问速度
  - where 子句涉及到的查询属性（ e.g. budget ）、连接属性 (dept_name)
  - group-by 子句中的分组属性 , e.g. building

```
select     building, avg (salary) as avg_salary
from       instructor natual_join department
where      budget>20000
group by building
    having avg (salary) > 42000
order by dept_name  desc
```

掌握：适宜建立索引的属性，索引对 select/update/insert/delete 的影响

- 对 delete-from-where 操作，如果在 where 查询条件属性上建立了索引，有利于在数据库文件中快速定位由 where 条件定义的需要删除的元组。
  但删除这些元组后，将引起 DBMS 将对索引进行调整重组，引起额外的系统开销，可能会降低 delete 的执行速度。
- 对 insert 操作，在关系表中插入新元组后， DBMS 将根据新插入的元组在索引属性上的值，对表中的索引进行调整重组，必然引起额外系统开销，降低 insert 的执行速度

掌握

- 对 update-set-where 操作，如果在 where 查询条件属性上建立了索引，有利于在数据库文件中快速定位 update 需要访问的元组。
  但如果 set 操作修改了索引涉及到的属性的值，则会引发 DBMS 对索引的重组。索引重组带来额外的系统开销，降低了 update 的执行速度

- E.g. 对关系表 Student(ID, name, dept, age) ，在属性 ID 上建立聚集或非聚集索引，该索引未必能提高下述 update 的执行速度

```
update   tbStudent
set      ID=ID +1
where    ID>20
```

例题

- **6.** Consider the following tables:

  factory(*factoryID*, *name*, *manager*, *account*)

  employee(*employeeID*, *name*, *age*, *factoryID*)

  airconditioner(*serialID*, *date*, *model*, *price*, *factoryID*)

- (1) Consider the following SQL query. In addition to the primary indices on the primary keys of the tables, on which attributes in the tables the indices can be further defined to speed up the query?

  select A.*factoryID*, avg(*employeeID*)
  from *factory* as A, *employee* as B
  where A.*factoryID* = B.*factoryID* and *age*>20 and *account*>30,000
  group by B.*factoryID*

- **答案**：以下属性上建立索引
  - *employee* 表的 *factoryID* ，利于加速 factory 和 employee 两表的连接操作，也有利于 group-by

    employee 表的 *age* ，利于从 *employee* 表中查找满足查询条件的目标元组

    factory 表的 *account* ，利于从 *factory* 表中查找满足查询条件的目标元组

例题

- (2) For the following query

  update *airconditioner*

  set *price* = *price* + 100

  where *model* =GL2020

  A nonclustered index is defined on the attribute *model*.

  Does this index speed up or slow down the operation, and why?

- 答案
  - model 上的索引将 speed up *update* 操作
  - DBMS 可以根据该索引，快速找到需要 update 的元组，修改元组中属性 price ，并且该索引无需调整

例题

- (3) For the following *delete* operation

  delete

  from *airconditioner*

  where *model*=HX8302 and *price*>1500

  A primary/clustered index has been defined on the primary key serialID. Does this index speed up or slow down the operation, and why? (4 points)

- 答案
  - 将 slow down delete 操作
  - 因为，根据 where 条件（涉及到 model 、 price 两个属性）查找需要删除的目标元组时，无法利用定义在主键 serialID 上的索引；并且，当删除满足 where 条件的目标元组后， DBMS 需要花额外时间重新调整定义在 serialID 上的主索引

    因此，整体上， serialID 上的主索引降低了 delete 操作速度

例题

- (4) if a multiple-attribute index (i.e. composite index) is defined on *airconditioner*(*serialID*, *date*, *model*, *price*, *factoryID*) by

  create index *DMFIndex* on *airconditioner*(*date*, *model*, *factoryID*)

  can the index *DMFIndex* speed up the following queries ， why?

  (i) select *serialID*

  from *airconditioner*

  where date='2021-11-30' and model='Haier'

  where 条件使用了 DMFIndex 的两个属性，且满足左前缀，索引起作用，可加速查询

  (ii) select *serialID*

  from *airconditioner*

  where date='2021-11-30' and price>2000

  where 条件使用了 DMFIndex 的一个属性 date ，且满足左前缀，索引起作用，可加速查询

如果查询条件改为：
where model='Haier' and price>2000
，索引不起作用

例题

(ii) select *serialID*

from *airconditioner*

where date='2021-11-30' and factoryID=1040

where 条件使用了 DMFIndex 的两个属性 date 和 factoryID ，从左前缀要求角度，只有查询条件 date= ' 2021-11-30 '会借助索引，索引能够部分加速查询

(IV) select *serialID*

from *airconditioner*

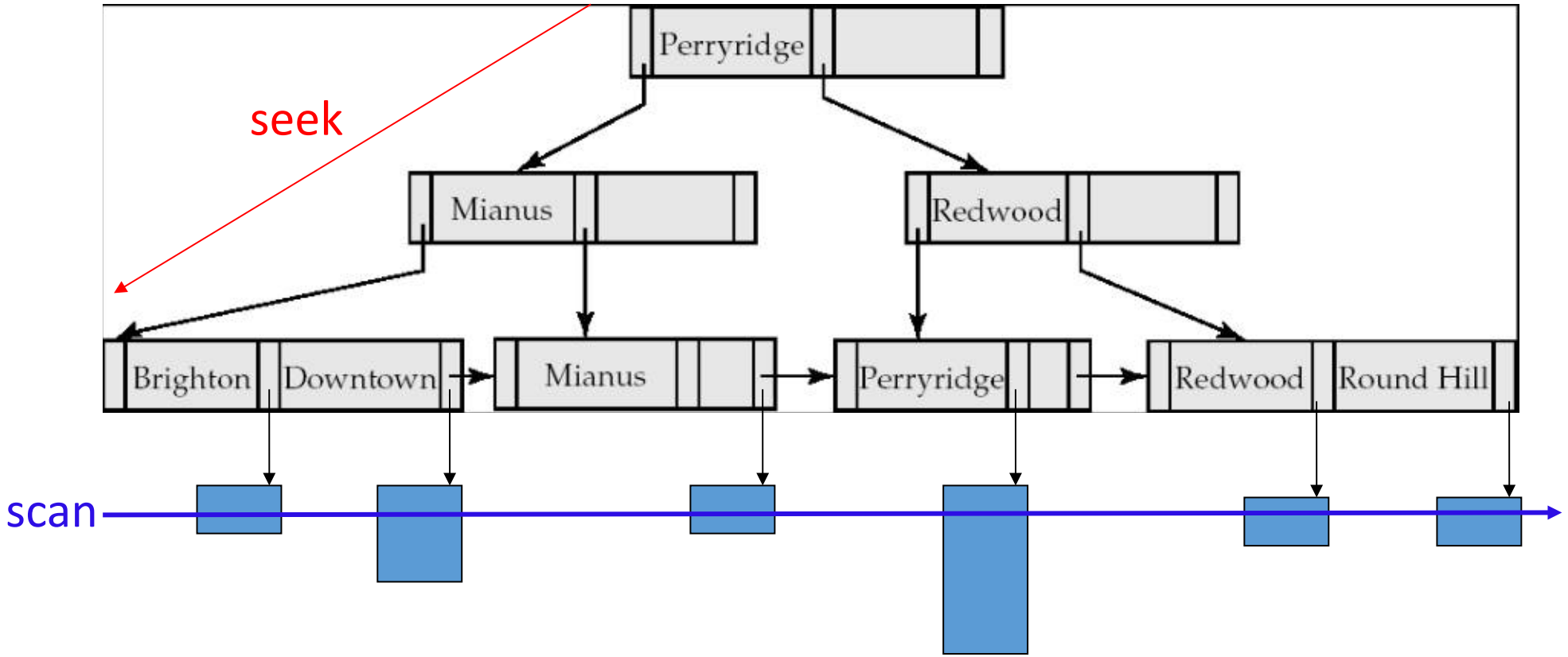
where model='Haier' and factoryID=1040

where 条件使用了 DMFIndex 的两个属性 model 和 factoryID ，但不满足左前缀，索引不起作用，无法加速查询

聚集索引树：索引 + **数据**

访问方式：

1. 聚集索引查找 seek ， e.g. where branch-name= ‘ Brighton ’，二分查找
2. 聚集索引扫描 scan， e.g. where account-number=10485 ，线性扫描



Thanks for your attention